

rHYPE Delta Neutral Trading System

Python Security Audit

Prepared by ShellBoxes

September 17th, 2025 - September 24th, 2025

Shellboxes.com

contact@shellboxes.com

Document Properties

Client	Raga Finance
Version	1.0
Classification	Public

Scope

Repository	Commit Hash
https://github.com/Nexus-2023/DeltaNeutralOracle	a1ba78308315e024878e8b81802dec8435e2b6f6

Re-Audit

Repository	Commit Hash
https://github.com/Nexus-2023/DeltaNeutralOracle	c9fb9066d155948098d8454b3f655bf45e3c3528

Contacts

COMPANY	EMAIL
ShellBoxes	contact@shellboxes.com

Contents

1 Introduction 5
1.1 About Raga Finance . . . 5
1.2 Approach & Methodology . . . 5
1.2.1 Risk Methodology . . . 6

2 Findings Overview 7
2.1 Summary . . . 7
2.2 Key Findings . . . 7

3 Finding Details 10
SHB.1 Mark/Oracle price unit mismatch . . . 10
SHB.2 Deposit S5: Shorts placed before EVM swap; swap failure leaves unhedged net short . . . 11
SHB.3 No concurrency control around state execution . . . 12
SHB.4 Unbounded recursive retries reset to S1 . . . 14
SHB.5 Balance-only finalization in spot conversions . . . 15
SHB.6 Floor to float rounding can produce zero transfers . . . 16
SHB.7 BlockchainConnectionManager not thread-safe; shared state without locks 17
SHB.8 Mutable default arguments in EpochService.create_epoch . . . 18
SHB.9 Order dedup hazard: client order id (CLOID) is constant 0 for all spot/perp orders . . . 19
SHB.10 Non-atomic nonce with RPC fallback . . . 21
SHB.11 Tight/infinite loops on `delta == 0` (Deposit S5 & Withdrawal S2) . . . 22
SHB.12 Price/balance helpers can return `None`; no freshness/sanity checks; callers assume numeric . . . 23
SHB.13 Withdrawal S6 mixes ints and floats (and may go negative) raw result sent on-chain . . . 24
SHB.14 Half-target split uses float division `int(ctx.target / 2)` . . . 25
SHB.15 API response parsing loses position sign; schema validation is minimal . . 26
SHB.16 Hedging chunks can stall when residuals < threshold and `len(chunks) > 0` 27
SHB.17 Private key loaded from env and retained in process memory . . . 28
SHB.18 Main loop has concurrency gaps and misleading operational logs . . . 29
SHB.19 General float usage in monetary math (binary rounding risk) . . . 30

SHB.20 Logs disclose exact balances, deltas, and swap/settlement amounts . . . 31
SHB.21 Misleading thresholds and sleep messages in main . . . 32
SHB.22 Rounding helper docstrings don't match return values . . . 33
SHB.23 SIGINT message says "continue until DONE" but code breaks early . . . 34

4 Best Practices 36
BP.1 Use integer division and `divmod` for target splits . . . 36
BP.2 Keep all monetary math in integers/Decimal – remove float+floor conversions . . . 37
BP.3 Guard against zero-amount transfers and clear dust deterministically . . . 38
BP.4 Reuse the same signed transaction across RPC fallback . . . 39
BP.5 Serialize nonces per account to prevent collisions . . . 40
BP.6 Estimate gas before sending and apply a bounded safety margin . . . 40
BP.7 Generate unique client order IDs (CLOIDs) and deduplicate orders . . . 41
BP.8 Avoid re-adding temporary ABIs on every price read . . . 42
BP.9 Replace recursive retries with bounded iterative backoff . . . 43
BP.10 Make BlockchainConnectionManager truly thread-safe . . . 43
BP.11 Correct scaling helpers' types and semantics . . . 44
BP.12 Hoist constants like Q192 to module scope . . . 45
BP.13 Cache environment reads and avoid per-iteration lookups . . . 45
BP.14 Cancel or skip duplicate open orders instead of re-placing . . . 46
BP.15 Introduce state-level DB "in-progress" claims before side effects . . . 47
BP.16 Back off when no fill progress to avoid tight loops and API hammering . . . 47
BP.17 Unify price unit conversions (Mark/Oracle in E8) and assert expected magnitudes . . . 48
BP.18 Fix mutable default arguments in epoch creation . . . 49
BP.19 Align logs with behavior and reduce sensitive data at INFO level . . . 50

5 Conclusion 51

6 Scope Files 52
6.1 Audit . . . 52
6.2 Re-Audit . . . 54

7 Disclaimer 57

1 Introduction

Raga Finance engaged ShellBoxes to conduct a security assessment on the rHYPE Delta Neutral Trading System beginning on September 17th, 2025 and ending September 24th, 2025. In this report, we detail our methodical approach to evaluate potential security issues associated with the implementation of smart contracts, by exposing possible semantic discrepancies between the smart contract code and design document, and by recommending additional ideas to optimize the existing code. Our findings indicate that the current version of smart contracts can still be enhanced further due to the presence of many security and performance concerns.

This document summarizes the findings of our audit.

1.1 About Raga Finance

Raga Finance brings institutional-grade structured products to help DeFi users maximize returns. Their one-click vaults simplify access to advanced yield strategies like delta neutral, autocompounding, yield looping, and more

Issuer	Raga Finance
Website	`https://www.raga.finance`
Type	Solidity Smart Contract
Documentation	Raga Finance Docs
Audit Method	Whitebox

1.2 Approach & Methodology

ShellBoxes used a combination of manual and automated security testing to achieve a balance between efficiency, timeliness, practicability, and correctness within the audit's scope. While manual testing is advised for identifying problems in logic, procedure, and implementation, automated testing techniques help to expand the coverage of smart contracts and can quickly detect code that does not comply with security best practices.

1.2.1 Risk Methodology

Vulnerabilities or bugs identified by ShellBoxes are ranked using a risk assessment technique that considers both the LIKELIHOOD and IMPACT of a security incident. This framework is effective at conveying the features and consequences of technological vulnerabilities.

Its quantitative paradigm enables repeatable and precise measurement, while also revealing the underlying susceptibility characteristics that were used to calculate the Risk scores. A risk level will be assigned to each vulnerability on a scale of 5 to 1, with 5 indicating the greatest possibility or impact.

- Likelihood quantifies the probability of a certain vulnerability being discovered and exploited in the untamed.
- Impact quantifies the technical and economic costs of a successful attack.
- Severity indicates the risk's overall criticality.

Probability and impact are classified into three categories: H, M, and L, which correspond to high, medium, and low, respectively. Severity is determined by probability and impact and is categorized into four levels, namely Critical, High, Medium, and Low.

Impact	Likelihood: High	Likelihood: Medium	Likelihood: Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

2 Findings Overview

2.1 Summary

The following is a synopsis of our conclusions from our analysis of the rHYPE Delta Neutral Trading System implementation. During the first part of our audit, we examine the smart contract source code and run the codebase via a static code analyzer. The objective here is to find known coding problems statically and then manually check (reject or confirm) issues highlighted by the tool. Additionally, we check business logics, system processes, and DeFi-related components manually to identify potential hazards and/or defects.

2.2 Key Findings

In general, these smart contracts are well-designed and constructed, but their implementation might be improved by addressing the discovered flaws, which include , 3 high-severity, 13 medium-severity, 3 low-severity, 4 informational-severity vulnerabilities.

Vulnerabilities	Severity	Status
SHB.1. Mark/Oracle price unit mismatch	HIGH	Fixed
SHB.2. Deposit S5: Shorts placed before EVM swap; swap failure leaves unhedged net short	HIGH	Fixed
SHB.9. Order dedup hazard: client order id (CLOID) is constant 0 for all spot/perp orders	HIGH	Acknowledged
SHB.3. No concurrency control around state execution	MEDIUM	Fixed
SHB.4. Unbounded recursive retries reset to S1	MEDIUM	Fixed
SHB.5. Balance-only finalization in spot conversions	MEDIUM	Acknowledged
SHB.6. Floor to float rounding can produce zero transfers	MEDIUM	Fixed

SHB.7. BlockchainConnectionManager not thread-safe; shared state without locks	MEDIUM	Fixed
SHB.8. Mutable default arguments in EpochService.create_epoch	MEDIUM	Fixed
SHB.10. Non-atomic nonce with RPC fallback	MEDIUM	Fixed
SHB.13. Withdrawal S6 mixes ints and floats (and may go negative) raw result sent on-chain	MEDIUM	Fixed
SHB.14. Half-target split uses float division `int(ctx.target / 2)`	MEDIUM	Fixed
SHB.15. API response parsing loses position sign; schema validation is minimal	MEDIUM	Acknowledged
SHB.16. Hedging chunks can stall when residuals < threshold and `len(chunks) > 0`	MEDIUM	Fixed
SHB.17. Private key loaded from env and retained in process memory	MEDIUM	Acknowledged
SHB.18. Main loop has concurrency gaps and misleading operational logs	MEDIUM	Fixed
SHB.11. Tight/infinite loops on `delta == 0` (Deposit S5 & Withdrawal S2)	LOW	Fixed
SHB.12. Price/balance helpers can return `None`; no freshness/sanity checks; callers assume numeric	LOW	Fixed
SHB.19. General float usage in monetary math (binary rounding risk)	LOW	Acknowledged
SHB.20. Logs disclose exact balances, deltas, and swap/settlement amounts	INFORMATIONAL	Acknowledged

SHB.21. Misleading thresholds and sleep messages in main	INFORMATIONAL	Acknowledged
SHB.22. Rounding helper docstrings don't match return values	INFORMATIONAL	Acknowledged
SHB.23. SIGINT message says "continue until DONE" but code breaks early	INFORMATIONAL	Acknowledged

3 Finding Details

SHB.1 Mark/Oracle price unit mismatch

- Severity: HIGH
- Likelihood: 2
- Status: Fixed
- Impact: 3

Description:

The read client documents and returns mark/oracle prices in E8, but the price feed decodes them using `scale_e4_to_decimal` (1e4), introducing a 10,000× error that propagates into order sizing and swap bounds.

Files Affected:

SHB.1.1: E8 mark price returned, decoded as E4

```
# blockchain/coreReadContract.py
def get_mark_price(self, index: int) -> int:
    """
    Get mark price for an asset.
    ...
    Returns:
        Mark price in E8 format
    """
    try:
        return self.client.call_contract_function(
            self.contract_address,
            "CORE_READ_CONTRACT",
            "markPx",
            index
        )
# utils/price_balance_util.py
```

```
def get_hype_mark_price(price_feed: PriceFeed = None) -> float:
    ...
    mark_price_e4 = price_feed.core_read_contract.get_mark_price(
        ↪ AssetIds.HYPE_PERP)
    mark_price_float = scale_e4_to_decimal(mark_price_e4)
    return mark_price_float
```

Recommendation:

Decode mark/oracle prices with scale_e8_to_decimal (1e8) and add assertions on expected price magnitudes. Audit all call sites using mark/oracle values (rounding helpers, chunk sizing, swap bounds) to keep units consistent.

Updates

The team resolved the issue by correcting the docstring, the current Mark Price is now in e4 decimals.

SHB.2 Deposit S5: Shorts placed before EVM swap; swap failure leaves unhedged net short

- Severity : HIGH
- Likelihood : 2
- Status : Fixed
- Impact : 3

Description:

Each chunk opens a short, then performs the EVM swap; swap failure raises `StateFatal` without a compensating close, leaving a net short.

Files Affected:

SHB.2.1: Open short -> swap; fatal on swap failure

```
# controller/Deposit/stateE.py
tx_hash, mark_px = self.delta_neutral_strategy_util.place_short_order(
    ↪ chunk[0])
...
tx_hash = self.core_contract.swap_usdt0_for_exact_hype(
    max_usdt0_in=scale_decimal_to_e6(evm_amount), amount_out_hype=
        ↪ scale_decimal_to_e18(delta)
)
...
if not receipt or status != 1:
    self.logger.error(f"EVM swap transaction failed: receipt={receipt},
        ↪ status={status}")
    raise StateFatal("Swap failed on evm")
```

Recommendation:

Swap-first sequencing with strict caps, or implement compensating logic to close/reduce the short on swap failure. Persist per-chunk progress so compensations are idempotent.

Updates

The team has solved the issue by downgrading the EVM swap failure from StateFatal to StateRetryable and adding a small retry counter around zero□fill orders.

SHB.3 No concurrency control around state execution

- Severity : MEDIUM
- Status : Fixed
- Likelihood : 2
- Impact : 2

Description:

If two runners can both see a state as incomplete and execute the same external action before completion is recorded. No DB lock or "in-progress" claim exists; completion is written after `is_finalized`.

Files Affected:

SHB.3.1: Run when not completed; completion recorded only after finalization

```
# controller/context_manager.py
if self._is_state_completed_in_db(ctx):
    ...
else:
    state_instance = self._get_state_instance(ctx.current_state)
    if state_instance.is_finalized(ctx):
        self._update_state_completion(ctx)
        self._move_to_next_state(ctx)
    else:
        state_instance.run(ctx)
```

Recommendation:

Introduce per-epoch/state "claim" (DB row lock or advisory lock) before side effects and embed idempotency keys in on-chain/API calls.

Updates

The team fixed the issue by adding a database-backed epoch claim and teaching the main loop to acquire and release that claim before running any states, which finally enforces a single active runner per epoch.

SHB.4 Unbounded recursive retries reset to S1

- Severity : MEDIUM
- Status : Fixed
- Likelihood : 2
- Impact : 2

Description:

On `StateRetryable` or any `Exception`, the engine calls `self.run(ctx)` recursively from S1, with no cap or backoff, risking duplicated financial actions and stack growth.

Files Affected:

SHB.4.1: Recursive restart without limits

```
# controller/context_manager.py
except StateRetryable as e:
    ...
    ctx.current_state = State.S1
    self.run(ctx)
...
except Exception as e:
    ...
    ctx.current_state = State.S1
    self.run(ctx)
```

Recommendation:

Replace recursion with an iterative loop using bounded retries and exponential backoff (see `retry_with_backoff` utility) and add idempotency for side-effecting calls.

Updates

The issue has effectively been addressed by replacing the recursive restart with an iterative loop, introducing retry_with_backoff to throttle repeated errors, and enforcing a MAX_RE-

TRIES cap that raises a fatal error once those retry attempts are exhausted.

SHB.5 Balance-only finalization in spot conversions

- Severity : MEDIUM
- Status : Acknowledged
- Likelihood : 2
- Impact : 2

Description:

Finalization depends only on balances; each loop re-submits a full-sized limit order if the balance hasn't dropped (e.g., resting orders), with no open-order reconciliation or deduplication.

Files Affected:

SHB.5.1: Re-placing orders based on balances only

```
# controller/Deposit/stateC.py
if usdt_balance <= TRADE_MARGIN:
    is_done = True
...
amount_to_be_converted_to_usdc = get_usdt_balance(self.price_feed)
tx_hash = self.delta_neutral_strategy_util.usdt_to_usdc(
    ↪ amount_to_be_converted_to_usdc)

# controller/Withdrawal/stateD.py
if usdc_balance <= TRADE_MARGIN:
    is_done = True
...
amount_to_be_converted_to_usdt = get_usdc_balance(self.price_feed)
tx_hash = self.delta_neutral_strategy_util.usdc_to_usdt(
    ↪ amount_to_be_converted_to_usdt)
```

Recommendation:

Track per-epoch client order IDs, fetch and reconcile open orders before placing new ones, and make order placement idempotent by (epoch,state,chunk).

Updates

The team acknowledged the risk and stated that finalization is based on the delta between the balance snapshots taken before and after placing an order. If the delta changes, the order has been executed. The code relies on a single source of truth: at any time, the USDT or USDC balance cannot exceed TRADE_MARGIN.

For the deposit flow: if USDT is transferred to Core, then all USDT must be transferred to the Perp account, since no other action occurs between S2 and S3. Therefore, resubmitting USDT orders is expected and aligns with the system's intent.

SHB.6 Floor to float rounding can produce zero transfers

- Severity : MEDIUM
- Likelihood : 2
- Status : Fixed
- Impact : 2

Description:

e6 amounts are converted to float, floored, then re-scaled; small dust can round to zero while finalization requires balances below margin thresholds, causing zero-amount calls in a loop.

Files Affected:

SHB.6.1: Float floor then re-scale before transfer

```
# controller/Deposit/stateD.py
amount_to_be_transferred_to_perp = get_usdc_balance(self.price_feed)
amount_usd = float(math.floor(scale_e6_to_decimal(
    ↪ amount_to_be_transferred_to_perp)))
```

```
amount_to_be_sent_to_perp_account = scale_decimal_to_e6(amount_usd)
tx_hash = self.core_contract.usd_class_transfer(
    ↪ amount_to_be_sent_to_perp_account, True)

# controller/Withdrawal/stateE.py
amount_to_be_transferred_to_evm = get_usdt_balance(self.price_feed)
amount_usd = float(math.floor(scale_e6_to_decimal(
    ↪ amount_to_be_transferred_to_evm)))
floor_amount_usd = math.floor(amount_usd)
scaled_amount = scale_decimal_to_e8(floor_amount_usd)
tx_hash = self.core_contract.transfer_usdt_to_evm(scaled_amount)
```

Recommendation:

Keep values in integers; transfer exact e6 balances (or ceil a dust-clearing tranche) and guard against zero-amount submits. Align MIN/TRADE margins with rounding policy.

Updates

The team resolved the issue and is no longer using floats. To ensure precision to three decimal places, they apply math.floor

SHB.7 BlockchainConnectionManager not thread-safe; shared state without locks

- Severity : MEDIUM
- Status : Fixed
- Likelihood : 2
- Impact : 2

Description:

Singleton uses `_lock = None` and no locking in `__new__`/`__init__`, exposing races in init/close and concurrent use of a shared client.

Files Affected:

SHB.7.1: Singleton without locking

```
# blockchain/connectionManager.py
class BlockchainConnectionManager:
    ...
    _instance: Optional['BlockchainConnectionManager'] = None
    _initialized: bool = False
    _lock = None
    def __new__(cls) -> 'BlockchainConnectionManager':
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
```

Recommendation:

Add a real process-level lock and serialize account nonces; prevent concurrent `send_transaction` without coordination.

Updates

The team resolved the thread-safety gap by guarding both __new__ and __init__ with a class-level lock (double-checked locking), so only one shared SmartContractClient is ever initialized even if multiple threads race on startup.

SHB.8 Mutable default arguments in EpochService.create_epoch

- Severity : MEDIUM
- Status : Fixed
- Likelihood : 2
- Impact : 2

Description:

Default list arguments are shared across calls, risking contamination of new epochs with prior state.

Files Affected:

SHB.8.1: Shared mutable defaults

```
# db/epoch_service.py
def create_epoch(...,
    short_price: List[float] = [],
    states_completed: List[bool] = [False, False, False, False, False,
        ↪ False, False]) -> Epoch:
    ...
```

Recommendation:

Use `None` defaults and instantiate fresh lists inside the function; audit DB rows for contamination.

Updates

They removed the mutable defaults in create_epoch; all list parameters now default to None, and the function instantiates fresh lists before inserting rows, eliminating cross-epoch state bleed.

SHB.9 Order dedup hazard: client order id (CLOID) is constant 0 for all spot/perp orders

- Severity: HIGH
- Status: Acknowledged
- Likelihood: 2
- Impact: 3

Description:

All order placements pass `client_order_id=0`, preventing deduplication and tracking on retries.

Files Affected:

SHB.9.1: Hard-coded client_order_id = 0

```
# utils/delta_neutral_strategy_util.py
price_e8, size_e8 = round_usdt_price_size(spot_price, amount_float)
tx_hash = self.place_limit_order(AssetIds.USDT0_SPOT, False, price_e8,
    ↪ size_e8, 0, False, 3)
...
cloid = 0
tx_hash = self.place_limit_order(AssetIds.USDT0_SPOT, True, price_e8,
    ↪ size_e8, cloid, False, 3)
...
cloid = 0
tx_hash = self.place_limit_order(AssetIds.HYPE_PERP, False, price_e8,
    ↪ size_e8, cloid, False, 3)
...
cloid = 0
tx_hash = self.place_limit_order(AssetIds.HYPE_PERP, True, price_e8,
    ↪ size_e8, cloid, True, 3)
```

Recommendation:

Generate per-epoch/state/segment CLOIDs and persist them; reconcile open orders before re-issuing.

Updates

The team acknowledged the risk and stated they are now using the current epoch as a unique CLOID.

SHB.10 Non-atomic nonce with RPC fallback

- Severity: MEDIUM
- Likelihood: 2
- Status: Fixed
- Impact: 2

Description:

Nonce obtained from “pending” each send, and the callable is re-executed under fallback; if first broadcast succeeded but was treated as failed, a second transaction can be built/sent with the next nonce.

Files Affected:

SHB.10.1: Pending nonce + callable re-execution under fallback

```
# blockchain/smartContractClient.py
def send_transaction(...):
    def _send_transaction():
        ...
        tx_params = {
            'from': self.address,
            'nonce': w3.eth.get_transaction_count(self.address, 'pending
                ↪ '),
            ...
        }
        ...
        tx_hash = w3.eth.send_raw_transaction(signed_tx.rawTransaction)
        return tx_hash.hex()
    return self.rpc_manager.execute_with_fallback(_send_transaction)
```

Recommendation:

Serialize nonces per account and re-send the same signed blob across endpoints on fallback; add on-chain idempotency.

Updates

The team resolved the issue. The smart contract client now caches the signed transaction and reuses that exact raw payload when falling back to secondary RPC endpoints, so nonce reuse is guaranteed and no duplicate broadcasts are constructed.

SHB.11 Tight/infinite loops on `delta == 0` (Deposit S5 & Withdrawal S2)

- Severity : LOW
- Status : Fixed
- Likelihood : 1
- Impact : 2

Description:

The inner loops `continue` on `delta == 0` with no backoff/exit, hammering APIs.

Files Affected:

SHB.11.1: Unbounded loop on no-fill

```
# controller/Deposit/stateE.py
if delta == 0:
    continue
# controller/Withdrawal/stateB.py
if delta == 0:
    continue
```

Recommendation:

Add bounded attempts, exponential backoff/jitter, and a “no progress” detector that exits or escalates.

Updates

The team resolved the delta==0 hammering by adding a retry counter with exponential backoff around the order-fill check: after each zero-fill, the state sleeps (doubling up to 30s) and gives up after five attempts, so the loop exits cleanly instead of spinning forever.

SHB.12 Price/balance helpers can return `None`; no freshness/sanity checks; callers assume numeric

- Severity : LOW
- Status : Fixed
- Likelihood : 1
- Impact : 2

Description:

Helpers swallow exceptions and return `None`; states immediately compare/multiply results with no timestamp or bounds validation.

Files Affected:

SHB.12.1: Helpers return None; states use without validation

```
# utils/price_balance_util.py
except Exception as e:
    logger.error(f"Error getting HYPE mark price: {str(e)}")
    return None
...
except Exception as e:
    logger.error(f"Error getting USDT balance: {str(e)}")
    return None

# controller/Deposit/stateB.py
usdt_balance = get_usdt_balance(self.price_feed)
if usdt_balance >= target_half - MIN_MARGIN:
```

```
13      is_done = True
```

Recommendation:

Validate non-`None` returns and add freshness/outlier checks (e.g., heartbeat windows, deviation caps). Treat invalid data as retryable with bounded backoff.

Updates

The helper functions now raise on RPC errors instead of returning None, so callers no longer run arithmetic on missing values.

SHB.13 Withdrawal S6 mixes ints and floats (and may go negative) raw result sent on-chain

- Severity : MEDIUM
- Status : Fixed
- Likelihood : 1
- Impact : 3

Description:

`withdrawal_amount` is computed by adding contract integers to float balances and immediately passed to `processEpochAndNotify` without normalization or non-negativity bounds.

Files Affected:

SHB.13.1: Mixed-type settlement forwarded directly

```
1 # controller/Withdrawal/stateF.py
2 withdrawal_amount = epoch_data.withdrawal_amount + amount_swapped_on_evm
      ↪  + amount_longed_on_perp - usdt_balance - usdc_balance - 2*
      ↪ MIN_MARGIN
3 ...
```

```
4 tx_hash = self.core_contract.process_epoch_and_notify(ctx.epoch,
    ↪ withdrawal_amount, 0)
```

Recommendation:

Normalize all operands to integer e6, clamp at zero, and assert invariants before calling the contract; log discrepancies for reconciliation.

Updates

The team partially addressed SHB□16 by normalizing the USDT transfer in Withdrawal S5: they convert the e6 balance to a float, floor it to the nearest mill, and then scale it back to e6/e8 before submitting the transfer. That guards one dust path, but Withdrawal S6 still mixes floats and ints and forwards the cast result without clamping, so the settlement amount can drift or go negative.

SHB.14 Half-target split uses float division `int(ctx.target / 2)`

- Severity : MEDIUM
- Likelihood : 1
- Status : Fixed
- Impact : 3

Description:

Target split casts through float then floors, losing precision for odd micro-units and risking drift.

Files Affected:

SHB.14.1: Float division before int cast

```
1 # controller/Deposit/stateA.py
2 amount_to_be_reserved_on_evm = int(ctx.target/2)
```

```
# controller/Withdrawal/stateA.py
amount_to_be_reserved_on_evm = int(ctx.target/2)
# controller/Deposit/stateB.py
amount_to_be_sent_to_core = int(ctx.target / 2)
```

Recommendation:

Use integer division (`ctx.target //2`) and persist the remainder for reconciliation (`q, r ↪ = divmod(ctx.target, 2)`).

Updates

The team has resolved the issue. Calls now use ctx.target // 2.

SHB.15 API response parsing loses position sign; schema validation is minimal

- Severity: MEDIUM
- Likelihood: 1
- Status: Acknowledged
- Impact: 3

Description:

Perp size and notional are wrapped in `abs(...)`, discarding direction; response schemas aren't fully validated.

Files Affected:

SHB.15.1: Sign discarded by abs(...)

```
# apis/fetch_perpetual_account_summary.py
def get_position_value(account_summary: Dict[str, Any]) -> float:
    ...
    return abs(position_value)
```

```
...
def account_position(account_summary: Dict[str, Any]) -> float:
    ...
    return abs(float(szi))
```

Recommendation:

Preserve sign for `szi` and notional; validate response structures and timestamps, failing closed on invalid data.

Updates

The team acknowledged the risk and stated that short orders will always be placed; therefore, the position will be 0 or negative. When placing long orders, reduceOnly = true is set, which prevents any long order that could create a positive position in the perp account. Maintaining this invariant simplifies the calculations.

SHB.16 Hedging chunks can stall when residuals < threshold and `len(chunks) > 0`

- Severity: MEDIUM
- Likelihood: 1
- Status: Fixed
- Impact: 3

Description:

`create_chunks` can return `[(0, delta)]` while `remaining_delta <= REMAINING_EXIT_SIZE`; `is_finalized` checks only `len(self.chunks)== 0`, and `run` iterates `while chunk[0] > ... or chunk[1] > ...`. A sub-threshold chunk leaves the state non-finalized yet unprocessed.

Files Affected:

SHB.16.1: Sub-threshold chunk + finalize on non-empty list

```
# utils/delta_neutral_strategy_util.py
if filled_perp_size > filled_evm_size:
    delta = filled_perp_size - filled_evm_size
    chunks.append((0, delta))
...
if remaining_delta <= REMAINING_EXIT_SIZE:
    return chunks

# controller/Deposit/stateE.py
if len(self.chunks) == 0:
    is_done = True
...
while chunk[0] > REMAINING_EXIT_SIZE or chunk[1] > REMAINING_EXIT_SIZE:
    ...
```

Recommendation:

Drop or auto-zero sub-threshold chunks (treat as finalized) or alter `is_finalized` to ignore sub-threshold entries; add an explicit cleanup pass.

Updates

The team has resolved the issue. create$_{c}hunksnowfiltersoutanysub-thresholdresidualsineveryreturnpath,sowhentheremainingdeltadropsbelowREMAINING_{E}XIT_{S}IZE,$

SHB.17 Private key loaded from env and retained in process memory

- Severity: MEDIUM
- Likelihood: 1
- Status: Acknowledged
- Impact: 3

Description:

The client stores `self.private_key` and holds a long-lived `Account` in memory; no secure enclave, encryption, or zeroization.

Files Affected:

SHB.17.1: Private key retained; account derived

```
# blockchain/smartContractClient.py
self.private_key = private_key
...
self.account: LocalAccount = Account.from_key(self.private_key)
```

Recommendation:

Use HSM/keystore integration and limit key lifetime in memory; wipe buffers after use and restrict environment access.

Updates

The team acknowledged the risk and stated that the oracle will run inside a TEE, significantly reducing exposure to external threats.

SHB.18 Main loop has concurrency gaps and misleading operational logs

- Severity : MEDIUM
- Status : Fixed
- Likelihood : 1
- Impact : 3

Description:

Epoch creation/claim lacks a lock; logs say “Sleeping for 30 minutes” but sleeps 10 seconds.

Files Affected:

SHB.18.1: Sleep/log mismatch; no locking around epoch create/run

```
# main.py
logger.info(f"Sleeping for 30 minutes")
time.sleep(10)
...
EpochService.create_epoch(epoch_onchain, target, withdrawal_amount,
    ↪ is_short, short_size_filled, short_price, states_completed)
```

Recommendation:

Correct logs and introduce DB-level uniqueness + atomic "claim" semantics (transactional row lock/advisory lock) before running an epoch.

Updates

The team resolved the issue. Epoch-level claims are now enforced through EpochService.claim_epoch, and the main loop acquires/releases that claim before running states, so only one runner processes a given epoch at a time and the "sleep 30 minutes" log now matches the actual 1800 second delay.

SHB.19 General float usage in monetary math (binary rounding risk)

- Severity : LOW
- Status : Acknowledged
- Likelihood : 1
- Impact : 2

Description:

Helpers convert between ints and floats, and several states compute with floats then cast, risking rounding drift.

Files Affected:

SHB.19.1: Float conversions in scaling helpers

```
# utils/scale_util.py
def scale_e6_to_decimal(usd_amount_int: int) -> float:
    return usd_amount_int / USD1E6
def scale_decimal_to_e6(usd_amount: float) -> int:
    return int(usd_amount * USD1E6)
```

Recommendation:

Keep arithmetic in integers (e.g., e6/e18) or `Decimal` with explicit quantization; minimize float round-trips.

Updates

The team acknowledged the issue and stated that the Hyperliquid API sends responses in decimals; therefore, they cannot fully switch to integer-based calculations.

SHB.20 Logs disclose exact balances, deltas, and swap/settlement amounts

- Severity : INFORMATIONAL
- Status : Acknowledged
- Likelihood : 0
- Impact : 3

Description:

INFO logs print sensitive balances and strategy sizes.

Files Affected:

SHB.20.1: Sensitive values logged at INFO

```
# controller/Deposit/stateB.py
self.logger.info(f"Deposit S2 finalized: usdt_balance {usdt_balance} >=
    ↪ target/2 {target_half}")
# controller/Withdrawal/stateF.py
self.logger.info(f"Calculation inputs: amount_swapped={
    ↪ amount_swapped_on_evm}, amount_longed={amount_longed_on_perp},
    ↪ usdt_balance={usdt_balance}, usdc_balance={usdc_balance}")
```

Recommendation:

Redact or downgrade to DEBUG; use structured logging with masking and access controls.

Updates

The team acknowledged the issue and stated that these values can generally be fetched from smart contracts as well; therefore, there is no need for masking.

SHB.21 Misleading thresholds and sleep messages in main

- Severity : INFORMATIONAL
- Likelihood : 0
- Status : Acknowledged
- Impact : 3

Description:

Log claims < 300000000 while code checks < 30000000; "Sleeping for 30 minutes" followed by 10s sleep.

Files Affected:

SHB.21.1: Threshold/sleep log mismatches

```
# main.py
```

```
if withdrawal_amount < 5000000 and target < 30000000:
    logger.info(f"Epoch not eligible ... target={target} < 300000000")
...
logger.info(f"Sleeping for 30 minutes")
time.sleep(10)
```

Recommendation:

Fix log strings to match code and actual sleep behavior to avoid operator confusion.

Updates

The team acknowledged the risk and stated that the code is currently in the testing phase.

SHB.22 Rounding helper docstrings don't match return values

- Severity : INFORMATIONAL
- Status : Acknowledged
- Likelihood : 0
- Impact : 3

Description:

Docstrings say the functions return a 4 tuple (rounded price/size and their E8 forms) but the functions actually return only the two E8 ints.

Files Affected:

SHB.22.1: Docstring claims 4-tuple; returns 2-tuple (ints)

```
def round_usdt_price_size(price: float, size: float) -> Tuple[float, int
    ↪ , float, int]:
    ...
    return price_e8, size_e8
```

```
4
5 def round_hype_perp_price_size(price: float, size: float) -> Tuple[float
      ↪ , int, float, int]:
6     ...
7     return price_e8, size_e8
```

Recommendation:

Align docstrings or change return signatures to the documented API to prevent misuse.

Updates

The team acknowledged the risk and stated that the code is currently in the testing phase.

SHB.23 SIGINT message says “continue until DONE” but code breaks early

- Severity: INFORMATIONAL
- Status: Acknowledged
- Likelihood: 0
- Impact: 3

Description:

When an epoch is still in progress and SIGINT is set, the code logs “Continuing until State.DONE...” then `break`s.

Files Affected:

SHB.23.1: Contradictory SIGINT behavior

```
1 # main.py
2 if sigint_received:
3     logger.info("SIGINT received but epoch not completed. Continuing
        ↪ until State.DONE...")
```

```
4       break
```

Recommendation:

Either honor the message (wait until DONE) or change the log to reflect an immediate, safe shutdown that records partial state explicitly.

Updates

The team acknowledged the risk and stated that the code is currently in the testing phase.

4 Best Practices

BP.1 Use integer division and `divmod` for target splits

Description:

Splitting the epoch target amount uses `int(ctx.target/2)`, which performs floating-point division then truncates. This can introduce precision loss and inconsistent accounting for odd micro-units.

Files Affected:

BP.1.1: controller/Deposit/stateA.py

```
amount_to_be_reserved_on_evm = int(ctx.target/2)
```

BP.1.2: controller/Withdrawal/stateA.py

```
amount_to_be_reserved_on_evm = int(ctx.target/2)
```

BP.1.3: controller/Deposit/stateB.py

```
target_half = int(ctx.target / 2)
...
amount_to_be_sent_to_core = int(ctx.target / 2)
```

Recommendation:

Use integer arithmetic end-to-end and preserve the remainder explicitly:

```
half, rem = divmod(ctx.target, 2) # both in e6 units
# persist 'rem' in epoch or reconcile in the last state
```

BP.2 Keep all monetary math in integers/Decimal – remove float+floor conversions

Description:

Multiple states convert e6-scaled integers to float, apply `math.floor`, then rescale. This can zero-out dust and cause stuck loops; it also introduces binary rounding risk.

Files Affected:

BP.2.1: controller/Deposit/stateD.py

```
amount_to_be_transferred_to_perp = get_usdc_balance(self.price_feed)
amount_usd = float(math.floor(scale_e6_to_decimal(
    ↪ amount_to_be_transferred_to_perp)))
amount_to_be_sent_to_perp_account = scale_decimal_to_e6(amount_usd)
tx_hash = self.core_contract.usd_class_transfer(
    ↪ amount_to_be_sent_to_perp_account, True)
```

BP.2.2: controller/Withdrawal/stateE.py

```
amount_to_be_transferred_to_evm = get_usdt_balance(self.price_feed)
amount_usd = float(math.floor(scale_e6_to_decimal(
    ↪ amount_to_be_transferred_to_evm)))
floor_amount_usd = math.floor(amount_usd)
scaled_amount = scale_decimal_to_e8(floor_amount_usd)
tx_hash = self.core_contract.transfer_usdt_to_evm(scaled_amount)
```

Recommendation:

Keep values in integers or `Decimal` with explicit quantization:

```
# integer path (preferred)
amount_e6 = get_usdc_balance(self.price_feed)
if amount_e6 > 0:
    tx_hash = self.core_contract.usd_class_transfer(amount_e6, True)
```

```
5
6 # or Decimal with quantize to 1e-6 before rescaling
```

BP.3 Guard against zero-amount transfers and clear dust deterministically

Description:

Floor-based conversions can produce zero transfer amounts while balances remain above thresholds, causing wasted gas and liveness issues.

Files Affected:

BP.3.1: controller/Deposit/stateD.py

```
1 amount_to_be_sent_to_perp_account = scale_decimal_to_e6(amount_usd)
2 tx_hash = self.core_contract.usd_class_transfer(
      ↪ amount_to_be_sent_to_perp_account, True)
```

Recommendation:

Add an explicit guard and a dust-clearing branch:

```
1 amt = get_usdc_balance(self.price_feed) # e6 integer
2 if amt == 0:
3     return # no-op
4 # If 0 < amt <= MIN_MARGIN, send exact amt once to avoid loop
5 tx_hash = self.core_contract.usd_class_transfer(amt, True)
```

BP.4 Reuse the same signed transaction across RPC fallback

Description:

`send_transaction` rebuilds the transaction and re-queries the 'pending' nonce on each fallback attempt, which can create duplicate transactions or nonce gaps.

Files Affected:

BP.4.1: blockchain/smartContractClient.py

```
def send_transaction(...):
    def _send_transaction():
        w3 = self._get_web3()
        ...
        tx_params = {'from': self.address,
                     'nonce': w3.eth.get_transaction_count(self.address, '
                        ↪ pending'),
                     'gas': gas_limit or self.gas_limit,
                     'chainId': w3.eth.chain_id}
        ...
        tx = func.build_transaction(tx_params)
        signed_tx = self.account.sign_transaction(tx)
        tx_hash = w3.eth.send_raw_transaction(signed_tx.rawTransaction)
        return tx_hash.hex()
    return self.rpc_manager.execute_with_fallback(_send_transaction)
```

Recommendation:

Build and sign once, then retry by broadcasting the same raw transaction to alternative RPCs:

```
signed = self.account.sign_transaction(tx)
def _broadcast(raw):
    w3 = self._get_web3()
```

```
4       return w3.eth.send_raw_transaction(raw).hex()
5   return self.rpc_manager.execute_with_fallback(lambda: _broadcast(signed.
        ↪ rawTransaction))
```

BP.5 Serialize nonces per account to prevent collisions

Description:

Using `'pending'` nonce without synchronization risks collisions when multiple threads/processes submit transactions concurrently.

Files Affected:

BP.5.1: blockchain/smartContractClient.py

```
1 'nonce': w3.eth.get_transaction_count(self.address, 'pending')
```

Recommendation:

Introduce a per-account nonce manager (process-wide mutex / DB row) that allocates the next nonce atomically and persists it until mined. Alternatively, queue txs through a single sender routine that serializes nonce usage.

BP.6 Estimate gas before sending and apply a bounded safety margin

Description:

Blindly setting a fixed gas limit can cause reverts (underestimation) or waste (overestimation). The client already exposes `estimate_gas` but call sites don't use it.

Files Affected:

BP.6.1: blockchain/smartContractClient.py

```
def estimate_gas(self, contract_address, abi_name, function_name, *args,
    ↪  value=None) -> int: ...
```

Recommendation:

For state-changing calls, preflight:

```
gas = self.estimate_gas(addr, abi, fn, *args, value=value)
tx_params['gas'] = int(gas * 1.10) # +10% headroom
```

BP.7 Generate unique client order IDs (CLOIDs) and deduplicate orders

Description:

All orders use `cloid = 0`, preventing idempotency and leading to duplicate resting orders on retries.

Files Affected:

BP.7.1: utils/delta_neutral_strategy_util.py

```
price_e8, size_e8 = round_usdt_price_size(spot_price, amount_float)
tx_hash = self.place_limit_order(AssetIds.USDT0_SPOT, True, price_e8,
    ↪ size_e8, cloid, False, 3)
...
cloid = 0
mark_px_float = get_hype_mark_price(self.price_feed)
price_e8, size_e8 = round_hype_perp_price_size(mark_px_float, size)
tx_hash = self.place_limit_order(AssetIds.HYPE_PERP, False, price_e8,
    ↪ size_e8, cloid, False, 3)
```

Recommendation:

Derive `cloid` as `f"{ctx.epoch}-{state}-{chunkId}"` and persist it. On retries, query by CLOID and skip or cancel existing orders before placing new ones.

BP.8 Avoid re-adding temporary ABIs on every price read

Description:

`get_spot_price` constructs and adds a temporary ABI name on each call, increasing overhead and complexity.

Files Affected:

BP.8.1: blockchain/coreReadContract.py

```
spot_price_abi = [{ "inputs": [{"internalType":"uint32","name":"index","
    ↪ type":"uint32"}],
                  "name":"spotPx","outputs":[{"internalType":"uint64","
                     ↪ name":"","type":"uint64"}],
                  "stateMutability":"view","type":"function"}]
self.client.add_abi("SPOT_PRICE_TEMP", spot_price_abi)
...
price = self.client.call_contract_function(ContractAddresses.
    ↪ SPOT_PRICE_TEMP_ADDRESS["address"],
                                           "SPOT_PRICE_TEMP", "spotPx", 166)
```

Recommendation:

Move the ABI to constants and register it once at startup; cache the contract instance and just call `spotPx` subsequently.

BP.9 Replace recursive retries with bounded iterative backoff

Description:

Recursive restarts on errors (`self.run(ctx)`) risk stack growth and repeated side effects.

Files Affected:

BP.9.1: controller/context_manager.py

```
except StateRetryable as e:
    ctx.current_state = State.S1
    self.run(ctx)
...
except Exception as e:
    ctx.current_state = State.S1
    self.run(ctx)
```

Recommendation:

Use an iterative loop with `retry_with_backoff` and state checkpoints to bound attempts and avoid re-executing external effects unnecessarily.

BP.10 Make BlockchainConnectionManager truly thread-safe

Description:

The singleton advertises thread safety but does not use a lock during initialization or access.

Files Affected:

BP.10.1: blockchain/connectionManager.py

```
class BlockchainConnectionManager:
    ...
    _lock = None
    def __new__(cls) -> 'BlockchainConnectionManager':
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
```

Recommendation:

Add a real `threading.Lock`, guard `__new__`/`__init__` with it, and prevent `close()` while clients are in-flight.

BP.11 Correct scaling helpers' types and semantics

Description:

`scale_e8_to_e6` is annotated to return `int` but uses `/` (float division). Mismatched types propagate to settlement math.

Files Affected:

BP.11.1: utils/scale_util.py

```
def scale_e8_to_e6(spot_price_int: int) -> int:
    """Convert integer spot price back to float"""
    return spot_price_int / PRICE1E2
```

Recommendation:

Fix return types (float) or use integer division where intended. Provide explicit `*_int` variants and keep a single representation through the pipeline to avoid accidental floats.

BP.12 Hoist constants like Q192 to module scope

Description:

`Q192` is recomputed inside the function on each call.

Files Affected:

BP.12.1: utils/price_balance_util.py

```
sqrt_price_x96 = price_feed.quote_contract.get_sqrt_price_x96()
Q192 = 2 ** 192
...
evm_price = num / den
```

Recommendation:

Define `Q192 = 2**192` at module scope and reuse it to reduce per-call overhead and clarify intent.

BP.13 Cache environment reads and avoid per-iteration lookups

Description:

Tight loops repeatedly call `os.getenv("USER_ADDRESS")` before every API request.

Files Affected:

BP.13.1: controller/Deposit/stateE.py

```
account_summary = fetch_perpetual_account_summary(os.getenv("
    ↪ USER_ADDRESS"))
```

Recommendation:

Resolve `USER_ADDRESS` once during initialization (or pass via context) and reuse it:

```
self.user = os.getenv("USER_ADDRESS")
...
account_summary = fetch_perpetual_account_summary(self.user)
```

BP.14 Cancel or skip duplicate open orders instead of re-placing

Description:

Balance-only finalization causes re-placement on every loop while previous orders are still resting.

Files Affected:

BP.14.1: controller/Deposit/stateC.py

```
if usdt_balance <= TRADE_MARGIN: ...
else:
    tx_hash = self.delta_neutral_strategy_util.usdt_to_usdc(
        ↪ amount_to_be_converted_to_usdc)
```

Recommendation:

Before placing, fetch open orders by CLOID or symbol and either (a) skip if an equivalent order exists, or (b) cancel+replace with updated sizing. This saves gas on redundant submissions and reduces book clutter.

BP.15 Introduce state-level DB "in-progress" claims before side effects

Description:

States execute external actions and only mark completion after `is_finalized`, leaving a crash window and no inter-process exclusion.

Files Affected:

BP.15.1: controller/context_manager.py

```
if self._is_state_completed_in_db(ctx):
    ...
else:
    state_instance = self._get_state_instance(ctx.current_state)
    if state_instance.is_finalized(ctx):
        self._update_state_completion(ctx)
    else:
        state_instance.run(ctx)
```

Recommendation:

Write an atomic "claimed_by / in_progress_at" flag in DB (or advisory lock) before `run`, then finalize/clear on success. This prevents duplicate transactions across runners and reduces wasted gas from duplicate submissions.

BP.16 Back off when no fill progress to avoid tight loops and API hammering

Description:

The per-chunk loop uses `continue` on `delta == 0` with no delay, causing hot loops.

Files Affected:

BP.16.1: controller/Deposit/stateE.py

```
if delta == 0:
    continue
```

BP.16.2: controller/Withdrawal/stateB.py

```
if delta == 0:
    continue
```

Recommendation:

Track attempts and add exponential backoff/jitter. Abort or alert after N non-progress iterations.

BP.17 Unify price unit conversions (Mark/Oracle in E8) and assert expected magnitudes

Description:

Mark and oracle prices are returned in E8, but downstream conversions assume E4 in some helpers, risking mis-sizing.

Files Affected:

BP.17.1: blockchain/coreReadContract.py

```
def get_mark_price(self, index: int) -> int:
    """... Mark price in E8 format"""
    return self.client.call_contract_function(..., "markPx", index)
```

BP.17.2: utils/price_balance_util.py

```
mark_price_e4 = price_feed.core_read_contract.get_mark_price(AssetIds.
    ↪ HYPE_PERP)
mark_price_float = scale_e4_to_decimal(mark_price_e4)
```

Recommendation:

Standardize on E8 throughout: rename variables `*_e8`, convert with `scale_e8_to_decimal`, and add asserts (e.g., 0.01 < price < 1e6) to catch unit mismatches early.

BP.18 Fix mutable default arguments in epoch creation

Description:

Default list objects are shared across calls, risking cross-epoch contamination.

Files Affected:

BP.18.1: db/epoch_service.py

```
def create_epoch(...,
    short_price: List[float] = [],
    states_completed: List[bool] = [False, False, False, False, False,
        ↪ False, False]) -> Epoch:
```

Recommendation:

Use `None` defaults and instantiate inside the function:

```
def create_epoch(..., short_price: Optional[List[float]] = None,
                 states_completed: Optional[List[bool]] = None) -> Epoch:
    short_price = short_price or []
    states_completed = states_completed or [False]*7
```

BP.19 Align logs with behavior and reduce sensitive data at INFO level

Description:

Mismatched messages (e.g., “Sleeping for 30 minutes” but sleeping 10s) and INFO logs leak balances and deltas.

Files Affected:

BP.19.1: main.py

```
logger.info(f"Sleeping for 30 minutes")
time.sleep(10)
```

Recommendation:

Make log messages accurate and downgrade sensitive numeric details to DEBUG with redaction or ranges. This reduces operational noise and avoids leaking strategy sizes.

5 Conclusion

We examined the design and implementation of rHYPE Delta Neutral Trading System in this audit and found several issues of various severities. We advise Raga Finance team to implement the recommendations contained in all 23 of our findings to further enhance the code's security. It is of utmost priority to start by addressing the most severe exploit discovered by the auditors then followed by the remaining exploits, and finally we will be conducting a re-audit following the implementation of the remediation plan contained in this report.

We would much appreciate any constructive feedback or suggestions regarding our methodology, audit findings, or potential scope gaps in this report.

6 Scope Files

6.1 Audit

Files	MD5 Hash
nexus-python/init.py	68b329da9893e34099c7d8ad5cb9c940
nexus-python/utils/delta_neutral_strategy_util.py	27cc00bdc60103c9a956efba60119ffb
nexus-python/utils/errors.py	e87efb51ee36bf98a57e19b5603b3bdc
nexus-python/utils/perp_price_size_util.py	25e2fed213411e33a4c3868a45fd625a
nexus-python/utils/price_balance_util.py	0d3bdbf40b0f44b5da325f318a63ad1e
nexus-python/utils/scale_util.py	91063fce2fc4d8815fe2f603879826ff
nexus-python/src/main.py	24844308ad20f8585aac255109dd0ecb
nexus-python/scripts/logReader.py	405abb6a7ff71714f1eceb7eec447d12
nexus-python/scripts/priceFetcher.py	db9816e84ec208630045955816c26583
nexus-python/migrations/env.py	a95d3383ad1086e2020daa015d1a8999
nexus-python/db/connection.py	75d49b1cd88e997af57d1a421827a1b1
nexus-python/db/epoch_service.py	03b8d69bb72cfa668af97a645c0fba43
nexus-python/db/schema.py	f7e0232fb27fd4434149acc00b6c6719
nexus-python/db/migrations/env.py	a95d3383ad1086e2020daa015d1a8999
nexus-python/db/migrations/versions/bc202c53f51e_initial_migration.py	2ef037969d9fff552b73936701627c06
nexus-python/controller/base_state.py	6695e8c8a849b89bc5fba730e6c2de58

nexus-python/controller/context_manager.py	3591f2148b159fafcc72236d0438cb25
nexus-python/controller/Withdrawal/stateA.py	15107f43cf7480e98fe2119c0b02d72b
nexus-python/controller/Withdrawal/stateB.py	62fd35fd890d82c360d38105a22b3e94
nexus-python/controller/Withdrawal/stateC.py	8740bc50f1f11eaa7569b8a6def437ae
nexus-python/controller/Withdrawal/stateD.py	64fd203b343dd34dcc50f9ca3e2bce1f
nexus-python/controller/Withdrawal/stateE.py	e7ec1369f794ac395669f63e42edc41b
nexus-python/controller/Withdrawal/stateF.py	69df7bd5252047ef7fdca69d3b6c665d
nexus-python/controller/Deposit/stateA.py	28c1904879bf14f1eeb933e5a9a9b230
nexus-python/controller/Deposit/stateB.py	d2e3638a4b81d8a0038f73a0cd2cffa0
nexus-python/controller/Deposit/stateC.py	470c058494fd2d8d41716219146c92d1
nexus-python/controller/Deposit/stateD.py	2d20f13e6fb58b406fbe3c844f30f6a0
nexus-python/controller/Deposit/stateE.py	d886e9acd0fd05a75f829db44339ba53
nexus-python/controller/Deposit/stateF.py	c6f11f44400b43878d793687a1c6ff76
nexus-python/blockchain/connectionManager.py	c0452a9564d0822b7c936bec9ce805bc
nexus-python/blockchain/constants.py	5e088bba248f6b6ba65413612ab598a3
nexus-python/blockchain/coreContract.py	93d35918860b2a47a3c7c9da5336adcc
nexus-python/blockchain/coreReadContract.py	d04842e1c6191aa768e0a34d7c892806
nexus-python/blockchain/quoteContract.py	af6f1e1740a1fc70dabf4d0be144f91e
nexus-python/blockchain/rpcManager.py	5d13ba26afd94e91e64cbb94890aef3d
nexus-python/blockchain/smartContractClient.py	ca451accd3169eb63a9e3047c55b0bb7

nexus-python/blockchain/__init__.py	b240a85b6b974b50c307773992d3c3ad
nexus-python/apis/connectionManager.py	5c4cc8fd82cb7660ad5201b939fd72f3
nexus-python/apis/fetch_perpetual_account_summary.py	6ed7a3afafe8c72ffaa5b4baf07669e2
nexus-python/apis/hyperliquid_info_client.py	75fd4f9623d73905277b1652803956f4
nexus-python/apis/__init__.py	7e0e8f909a12e5918cba4e2d6fbc1b0a

6.2 Re-Audit

Files	MD5 Hash
./init.py	68b329da9893e34099c7d8ad5cb9c940
./src/main.py	1e63320a70d5ce28ff711a637f12f010
./db/epoch_service.py	caabd6416769363ec11aedff4e445649
./db/connection.py	75d49b1cd88e997af57d1a421827a1b1
./db/schema.py	a634f70b34f526d20145b2823bbaaf0a
./db/migrations/env.py	a95d3383ad1086e2020daa015d1a8999
./db/migrations/versions/bc202c53f51e_initial_migration.py	ef7fb1ec07879557de292337dc5f8140
./blockchain/quoteContract.py	af6f1e1740a1fc70dabf4d0be144f91e
./blockchain/coreReadContract.py	2352e4f75ecdfa439ff7d8610d89758b
./blockchain/constants.py	1a2afd462f0156623c767980705acbb9
./blockchain/__init__.py	b240a85b6b974b50c307773992d3c3ad

./blockchain/connectionManager.py	8db132e6e123e7d1d6641fc7ca1b21c4
./blockchain/smartContractClient.py	27cc87dbf01f9692332e52c579fa44ab
./blockchain/coreContract.py	53ecab34103b8f7b4fce6c3947c7514e
./blockchain/rpcManager.py	a9fca811d5b9cce27689b89864dcc8af
./controller/base_state.py	6695e8c8a849b89bc5fba730e6c2de58
./controller/context_manager.py	6228d20fa82d966380f27bb143f9bd82
./controller/Withdrawal/stateC.py	1ff7d60f1a45fae85cc91273deea9181
./controller/Withdrawal/stateF.py	c3885bf2bbb777ccb83a904d5569b69f
./controller/Withdrawal/stateB.py	ccd5bc387bdd2cd6d73a0c46bdb70950
./controller/Withdrawal/stateE.py	7add2b03fe2484da6e70cd51c35f35b3
./controller/Withdrawal/stateA.py	466abb57b5aa661382e4bdf28be292a6
./controller/Withdrawal/stateD.py	64fd203b343dd34dcc50f9ca3e2bce1f
./controller/Withdrawal/__pycache__/stateC.cpython-39.pyc	2d1939dff70bcea7da0204ccb5c693b1
./controller/Withdrawal/__pycache__/stateD.cpython-39.pyc	2d1b547e4029636189702bd72888511e
./controller/Withdrawal/__pycache__/stateE.cpython-39.pyc	18f6cdf4adf1e3888161ab545ec6ac62
./controller/Withdrawal/__pycache__/stateB.cpython-39.pyc	a1e01be33c6c4e12a671d51cdc6fcead
./controller/Withdrawal/__pycache__/stateF.cpython-39.pyc	0eeccbb93eaa3504b74601e38c32118c
./controller/Withdrawal/__pycache__/stateA.cpython-39.pyc	cbcb77dc2ba75051c63d914058bbe2e1

./controller/Deposit/stateC.py	470c058494fd2d8d41716219146c92d1
./controller/Deposit/stateF.py	c6f11f44400b43878d793687a1c6ff76
./controller/Deposit/stateB.py	dde9fa8b956ad9b57b2919b63f5b4d1b
./controller/Deposit/stateE.py	b339c5bbc7e2a11a8e4ff0c9d1a4518a
./controller/Deposit/stateA.py	4d44df0f2f00da9ea9085efc1405f722
./controller/Deposit/stateD.py	2d20f13e6fb58b406fbe3c844f30f6a0
./utils/logging_config.py	e875a073882a1ee8d9372db2ba8c800c
./utils/perp_price_size_util.py	25e2fed213411e33a4c3868a45fd625a
./utils/delta_neutral_strategy_util.py	882c73db484f78ab42d4c7e8962460f2
./utils/scale_util.py	a211d3672387142fb50256a7347e506a
./utils/errors.py	e87efb51ee36bf98a57e19b5603b3bdc
./utils/price_balance_util.py	fad1ebc3fe5641f5288e0e195e4a5a8b
./utils/telegram_notifier.py	311b94857a2b16c0ec97b08c511c7192
./apis/__init__.py	7e0e8f909a12e5918cba4e2d6fbc1b0a
./apis/connectionManager.py	5c4cc8fd82cb7660ad5201b939fd72f3
./apis/hyperliquid_info_client.py	75fd4f9623d73905277b1652803956f4
./apis/fetch_perpetual_account_summary.py	1af19bdf8447b3b975e5869dc7101aa5

7 Disclaimer

Shellboxes reports should not be construed as "endorsements" or "disapprovals" of particular teams or projects. These reports do not reflect the economics or value of any "product" or "asset" produced by any team or project that engages Shellboxes to do a security evaluation, nor should they be regarded as such. Shellboxes Reports do not provide any warranty or guarantee regarding the absolute bug-free nature of the examined technology, nor do they provide any indication of the technology's proprietors, business model, business or legal compliance. Shellboxes Reports should not be used in any way to decide whether to invest in or take part in a certain project. These reports don't offer any kind of investing advice and shouldn't be used that way. Shellboxes Reports are the result of a thorough auditing process designed to assist our clients in improving the quality of their code while lowering the significant risk posed by blockchain technology. According to Shellboxes, each business and person is in charge of their own due diligence and ongoing security. Shellboxes does not guarantee the security or functionality of the technology we agree to research; instead, our purpose is to assist in limiting the attack vectors and the high degree of variation associated with using new and evolving technologies.



For a Contract Audit, contact us at contact@shellboxes.com